proyecto

Ingeniería de Aplicaciones Web 2026 — Proyecto

Descripción General

Cada comisión de **4 o 5 integrantes** desarrollará un **ecosistema de aplicaciones web interconectadas**, inspirado en plataformas reales del mundo tecnológico. Cada integrante será responsable de desarrollar **una aplicación web completa e independiente**, y luego el equipo deberá integrarlas en un sistema cohesivo.

La propuesta busca simular el trabajo real en equipos de ingeniería de software, donde distintos desarrolladores trabajan en servicios distintos que deben comunicarse entre sí mediante **APIs REST**, cada uno gestionando su propia base de datos y sus propios dominios de responsabilidad.

El proyecto se divide en **tres etapas**:

1. **Planificación** — definición del sistema, distribución de responsabilidades y acuerdo sobre contratos de API.
2. **Implementación individual** — cada integrante desarrolla su aplicación web de forma aislada.
3. **Integración y aplicaciones globales** — las apps se conectan entre sí, y se construyen dos aplicaciones transversales: un **Control Plane** y un **Analytics Dashboard**.

Tipos de Proyecto

Cada comisión deberá elegir uno de los siguientes tipos de sistema a desarrollar. Los integrantes trabajan sobre el mismo tipo, cada uno responsable de una de las webapps que lo componen.

Junto con la elección del tipo, la comisión deberá definir un **dominio de aplicación específico** y una **marca propia** para su sistema. No alcanza con elegir “plataforma de delivery”: hay que decidir qué producto concreto se está construyendo (por ejemplo, una plataforma de delivery de comida saludable llamada *FreshRun*, o un marketplace de artículos de diseño llamado

Craftly). Esta identidad debe reflejarse en el nombre de los repositorios, el diseño visual y la documentación del proyecto.

🚗 Tipo A — Plataforma de Transporte (estilo Uber)

Classroom

Un sistema de transporte on-demand compuesto por cuatro aplicaciones (o cinco, para comisiones de 5 integrantes):

Webapp	Responsabilidad
Driver App	Interfaz para conductores: disponibilidad, aceptación de viajes, historial de viajes realizados.
Rider App	Interfaz para pasajeros: solicitud de viajes, seguimiento, historial y calificaciones.
Payments App	Gestión del flujo de pagos: cobros a pasajeros, liquidaciones a conductores, historial de transacciones.
Feedback App	Sistema de reseñas y calificaciones: calificación mutua entre pasajeros y conductores, moderación y reportes.
Promotions App ⚠️ *solo comisiones de 5 integrantes*	Gestión de promociones y fidelización: códigos de descuento, campañas promocionales, programa de puntos y beneficios para pasajeros frecuentes.

🍔 Tipo B — Plataforma de Delivery (estilo Pedidos Ya)

Classroom

Un sistema de pedidos y entrega a domicilio compuesto por cuatro aplicaciones (o cinco, para comisiones de 5 integrantes):

Webapp	Responsabilidad
Buyer App	Interfaz para compradores: exploración de restaurantes/tiendas, carrito, seguimiento de pedidos.
Seller App	Interfaz para vendedores/restaurantes: gestión de menú o catálogo, recepción y gestión de pedidos.
Delivery App	Interfaz para repartidores: asignación de pedidos, confirmación de retiro y entrega, historial.

Webapp	Responsabilidad
Payments App	Gestión de pagos: cobros a compradores, liquidaciones a vendedores y repartidores, historial de transacciones.
Feedback App ⚠️ *solo comisiones de 5 integrantes*	Sistema de reseñas y calificaciones: calificación de restaurantes/tiendas y repartidores por parte de los compradores, moderación de reseñas y reportes.

🛒 Tipo C — Marketplace (estilo Mercado Libre)

Classroom

Un marketplace de compra-venta entre usuarios, compuesto por cuatro aplicaciones (o cinco, para comisiones de 5 integrantes):

Webapp	Responsabilidad
Buyer App	Interfaz para compradores: búsqueda de productos, carrito, historial de compras y seguimiento.
Seller App	Interfaz para vendedores: publicación y gestión de productos, gestión de ventas y stock.
Shipping App	Interfaz para operadores logísticos: gestión de envíos, actualización de estados, historial de entregas.
Payments App	Gestión de pagos: cobros, acreditaciones a vendedores, historial de transacciones y disputas.
Feedback App ⚠️ *solo comisiones de 5 integrantes*	Sistema de reseñas y calificaciones: calificación de productos y vendedores por parte de los compradores, moderación de reseñas y reportes.

Etapa 1 — Planificación del Proyecto

Fecha de entrega: 27 de Abril de 2026
Fecha de defensa: 30 de Abril de 2026

Objetivo

Antes de escribir una sola línea de código, la comisión debe tener una comprensión sólida y consensuada de todo el sistema. Esta etapa es un ejercicio de **diseño de arquitectura y negociación de contratos**, habilidades centrales del trabajo en equipo de ingeniería.

Formato de entrega

La documentación se entrega en el repositorio de docs como archivos Markdown, con la siguiente estructura:

```
README.md                    ← índice general: nombre del proyecto, tipo, integran
docs/
  01-descripcion.md          ← entregable 1.1
  02-responsabilidades.md    ← entregable 1.2
  03-apis.md                 ← entregable 1.3
  04-modelo-de-datos.md      ← entregable 1.4
  05-usuarios.md             ← entregable 1.5
```

Entregables de la Etapa 1

1.1 — Descripción del sistema

Un documento que explique el sistema elegido en términos funcionales:

- ¿Qué problema resuelve?
- ¿Quiénes son los actores del sistema (usuarios finales de cada app, administradores)?
- ¿Cuál es el flujo principal de uso? (ej: un pasajero solicita un viaje → un conductor lo acepta → se procesa el pago → ambos se califican)

1.2 — Asignación de responsabilidades

Un cuadro que indique claramente:

- Qué webapp desarrolla cada integrante.
- Qué datos son propios de cada app (en qué base de datos vive cada entidad).
- Qué datos o acciones requieren comunicación con otra app (y a través de qué API).

1.3 — Diseño de APIs inter-servicios

Para cada punto de integración entre aplicaciones, documentar:

- **Endpoint:** método HTTP y ruta (ej: `POST /api/payments/charge`)
- **Request:** qué datos envía el llamador
- **Response:** qué datos responde el servicio
- **Quién llama a quién:** qué app consume qué endpoint de qué otra app

Este contrato de API debe estar acordado y firmado por todos los integrantes antes de comenzar la Etapa 2, ya que es la base sobre la que cada uno desarrollará de forma aislada.

1.4 — Modelo de datos por aplicación

Por cada webapp, un diagrama o descripción de las entidades principales de su base de datos (tablas, relaciones relevantes). No es necesario que sea un DER formal, pero sí que esté claro qué persiste cada app.

Los datos integrados pueden contener duplicados —como en el caso de los usuarios, entre otros— por lo que es necesario identificar las posibles inconsistencias y definir una estrategia para resolverlas.

1.5 — Usuarios compartidos

El sistema utiliza **Clerk como servicio centralizado de autenticación** para todas las aplicaciones. Esto significa que los usuarios se autentican a través de Clerk independientemente de qué app estén usando, y la identidad se propaga entre servicios mediante el token JWT emitido por Clerk.

En este punto, la comisión deberá definir:

- ¿Qué apps comparten usuarios? (ej: en Uber, un rider puede acceder a las apps de pagos, feedback y promociones)
- ¿Qué claims del token JWT son relevantes para cada app? (ej: roles, permisos, identificador de usuario compartido entre servicios)

Etapa 2 — Implementación Individual

Fecha de entrega: 28 de Mayo de 2026
Fechas de defensa: 1 y 4 de Junio de 2026

Objetivo

Cada integrante desarrolla **su webapp de forma completamente aislada**, como si fuera un producto independiente. Al finalizar esta etapa, cada app debe funcionar por sí sola, con datos de prueba propios, sin depender del funcionamiento real de las otras apps.

Las llamadas a APIs de otras webapps **deben mockearse** o simularse durante esta etapa. Lo importante es que los contratos definidos en la Etapa 1 estén respetados.

Stack tecnológico

Cada webapp deberá construirse con el siguiente stack tecnológico:

Capa	Tecnología
Frontend / Full-stack	**Next.js**
Base de datos	**PostgreSQL** (base de datos propia por app)

Capa	Tecnología
Autenticación	**Clerk** (servicio centralizado compartido por todas las apps)
Pagos (solo la Payments App)	**Mercado Pago** en modo sandbox
Estilos	Tailwind CSS, Chakra UI o Bootstrap
ORM	Prisma, Knex, o `pg` directamente
Deploy	**Vercel** (una instancia por app) + Railway / Supabase / Neon / Vercel Postgres

Requisitos de cada webapp individual

Cada webapp debe cumplir con los siguientes requisitos, adaptados a su dominio:

✅ **Páginas y componentes reutilizables** en Next.js.

✅ **API propia** — cada app expone sus propios endpoints REST (los cuales están pensados para las otras apps en la Etapa 3, pero pueden ser utilizados por su frontend de ser necesario).

✅ **Base de datos PostgreSQL propia** — cada app es dueña de sus datos.

✅ **Autenticación** — login/logout para usuarios administradores (obligatorio). Login para usuarios finales según corresponda al dominio de la app.

✅ **Panel de administración** — el usuario administrador debe poder gestionar los datos principales de la app y visualizar al menos un listado o reporte relevante.

✅ **Búsqueda y paginación** — donde aplique, implementar búsqueda y paginación con parámetros en la URL.

✅ **Manejo de errores** — errores generales y páginas 404.

✅ **Validación de formularios** del lado del servidor.

✅ **Accesibilidad** — aplicar buenas prácticas básicas.

✅ **Consumo de al menos una API externa** — integrar un servicio externo que aporte valor al dominio de la app. Debe hacerse un request real y procesarse la respuesta (no embeds). Las APIs de las otras webapps del mismo proyecto cuentan como externas a los fines de este requisito.

✅ **Integración con Mercado Pago** (solo para la Payments App) — flujo de pago en modo sandbox.

✅ **Opcional — IA** — se puede incorporar funcionalidad basada en inteligencia artificial (sugerencias, chatbot, descripciones automáticas, etc.). No es obligatorio, pero suma.

Variables de entorno y secretos

Cada app utiliza credenciales sensibles que **nunca deben commitearse al repositorio**: connection strings de la base de datos, claves de Clerk, credenciales de Mercado Pago, claves de APIs externas, etc.

- Usar un archivo `.env.local` para las variables de entorno en desarrollo local. Este archivo debe estar incluido en el `.gitignore`.
- En Vercel, configurar esas mismas variables desde el panel de configuración del proyecto (*Settings → Environment Variables*).
- Incluir en el repositorio un archivo `.env.example` con los nombres de las variables necesarias pero sin sus valores, para que cualquiera que clone el repo sepa qué configurar.

Nota sobre el aislamiento

Durante esta etapa, las apps no necesitan estar conectadas entre sí. Cada integrante trabaja en su propio repositorio, con su propio deploy. Si una app necesita datos de otra (ej: la Rider App necesita saber si un conductor está disponible), debe usar datos mockeados o un stub del endpoint esperado.

Entregables de la Etapa 2

Cada integrante deberá entregar, de acuerdo a la webapp que le fue asignada en la Etapa 1:

1. **Aplicación web funcional** — la webapp correspondiente, deployada en Vercel y accesible mediante un link de producción.
2. **Código fuente completo** — todo el código disponible en el repositorio designado para esa app, con historial de commits que refleje el desarrollo individual.
3. **Datos cargados** — la aplicación no puede estar vacía. Debe contar con datos relevantes precargados que permitan evaluarla sin necesidad de cargar información manualmente (ej: viajes realizados, pedidos en distintos estados, productos publicados, transacciones procesadas, reseñas cargadas).
4. **README** — breve y conciso, debe incluir: descripción de la app, link al deploy, y credenciales o instrucciones para acceder con cada tipo de usuario disponible (ej: administrador, usuario final). No debe ser extenso.

Etapa 3 — Integración y Aplicaciones Globales

Fecha de entrega: 25 de Junio de 2026
Fechas de defensa: 29 de Junio de 2026 y 2 de Julio de 2026

Objetivo

En esta etapa, las aplicaciones individuales se conectan entre sí y la comisión desarrolla **dos aplicaciones transversales** que operan sobre el sistema completo.

3.1 — Integración entre webapps

Reemplazar los mocks de la Etapa 2 por llamadas reales a los endpoints de las otras apps, respetando los contratos definidos en la Etapa 1. Se espera que al menos los flujos principales del sistema funcionen de punta a punta.

Ejemplos de flujos integrados:

- **Uber:** un pasajero solicita un viaje (Rider App) → un conductor lo acepta (Driver App) → al finalizar, se cobra el pago (Payments App) → ambos se califican (Feedback App).
- **Delivery:** un comprador hace un pedido (Buyer App) → el vendedor lo confirma (Seller App) → un repartidor lo retira y entrega (Delivery App) → se procesa el pago (Payments App).
- **Marketplace:** un comprador compra un producto (Buyer App) → el vendedor lo despacha (Seller App) → el operador gestiona el envío (Shipping App) → el pago se acredita al vendedor (Payments App).

3.2 — Control Plane

Una nueva webapp (desarrollada de forma colaborativa por la comisión) que actúa como **panel de administración global del sistema**. Permite a un superadministrador operar sobre todas las apps desde un único lugar.

Funcionalidades esperadas:

- **Visión consolidada** de las entidades principales de cada app (usuarios, transacciones, pedidos, etc.).
- **Acciones de gestión** sobre cualquiera de las apps: activar/desactivar usuarios, resolver disputas, gestionar configuraciones globales.
- **Comunicación con las APIs** de cada webapp individual.

No reemplaza los paneles de administración individuales de cada app, sino que los complementa con una vista de mayor nivel.

3.3 — Analytics Dashboard

Una segunda webapp nueva (también colaborativa) que presenta **métricas y reportes sobre el sistema completo**.

Funcionalidades esperadas:

- **Indicadores clave** del negocio: volumen de transacciones, usuarios activos, pedidos completados, ingresos, calificaciones promedio, etc.
- **Visualizaciones** (tablas, gráficos o indicadores) que permitan entender el estado del sistema de un vistazo.
- **Datos consolidados** obtenidos consultando las APIs de las webapps individuales.

El Analytics Dashboard no es un CRUD — es una herramienta de lectura y análisis. La complejidad está en consolidar datos de múltiples fuentes y presentarlos de manera útil.

Entregables de la Etapa 3

1. **Control Plane y Analytics Dashboard** — ambas aplicaciones deployadas en Vercel, con código completo en sus respectivos repositorios y un README breve con link al deploy e instrucciones para acceder con cada tipo de usuario.
2. **Aplicaciones individuales integradas** — las webapps de la Etapa 2 debidamente conectadas entre sí, con los mocks reemplazados por llamadas reales, de forma que al menos los flujos principales del sistema funcionen de punta a punta.
3. **Datos cargados** — el sistema completo debe contar con datos relevantes precargados que permitan recorrerlo y evaluarlo sin necesidad de cargar información manualmente (ej: viajes realizados, pedidos en distintos estados, transacciones procesadas).
4. **Al menos un flujo de punta a punta demostrable** en la defensa.

Estructura de repositorios

Cada comisión deberá tener **un repositorio por aplicación**, más un repositorio de documentación. El `[nombre]` es la marca propia elegida por la comisión al formar el team en GitHub Classroom (ej: `freshrun`, `craftly`), y es agregado automáticamente por Classroom al final del nombre de cada repositorio.

🚗 Tipo A

Repositorio	Contenido
`proyecto-a-docs-[nombre]`	Documentación de la Etapa 1
`proyecto-a-driver-[nombre]`	Driver App
`proyecto-a-rider-[nombre]`	Rider App

Repositorio	Contenido
`proyecto-a-payments-[nombre]`	Payments App
`proyecto-a-feedback-[nombre]`	Feedback App
`proyecto-a-promotions-[nombre]`	Promotions App *(solo comisiones de 5 integrantes)*
`proyecto-a-control-plane-[nombre]`	Control Plane (colaborativo)
`proyecto-a-analytics-[nombre]`	Analytics Dashboard (colaborativo)

🍔 Tipo B

Repositorio	Contenido
`proyecto-b-docs-[nombre]`	Documentación de la Etapa 1
`proyecto-b-buyer-[nombre]`	Buyer App
`proyecto-b-seller-[nombre]`	Seller App
`proyecto-b-delivery-[nombre]`	Delivery App
`proyecto-b-payments-[nombre]`	Payments App
`proyecto-b-feedback-[nombre]`	Feedback App *(solo comisiones de 5 integrantes)*
`proyecto-b-control-plane-[nombre]`	Control Plane (colaborativo)
`proyecto-b-analytics-[nombre]`	Analytics Dashboard (colaborativo)

🛒 Tipo C

Repositorio	Contenido
`proyecto-c-docs-[nombre]`	Documentación de la Etapa 1
`proyecto-c-buyer-[nombre]`	Buyer App
`proyecto-c-seller-[nombre]`	Seller App
`proyecto-c-shipping-[nombre]`	Shipping App
`proyecto-c-payments-[nombre]`	Payments App

Repositorio	Contenido
`proyecto-c-feedback-[nombre]`	Feedback App *(solo comisiones de 5 integrantes)*
`proyecto-c-control-plane-[nombre]`	Control Plane (colaborativo)
`proyecto-c-analytics-[nombre]`	Analytics Dashboard (colaborativo)

Criterios de Evaluación

Criterio	Descripción
Completitud individual	Cada webapp cumple los requisitos de la Etapa 2
Calidad de la integración	Los flujos inter-apps funcionan correctamente
Diseño de API	Los contratos definidos en la Etapa 1 son coherentes y se respetan
Control Plane y Analytics	Funcionalidad, utilidad y calidad de las apps globales
Calidad del código	Organización, legibilidad y buenas prácticas en todos los repos
Diseño y UX	Atractivo visual y facilidad de uso
Defensa	Capacidad de explicar decisiones técnicas y de diseño

Dinámica de Trabajo

- Cada integrante es **dueño y responsable** de su webapp. El éxito individual depende de él/ella.
- Las apps globales (Control Plane y Analytics) son **responsabilidad colectiva** de la comisión.
- La Etapa 1 es el contrato del equipo — cambios posteriores a los acuerdos deben ser consensuados por todos. **Todo cambio debe quedar reflejado en el repositorio de documentación** mediante un branch, un Pull Request y su correspondiente merge, donde se explique claramente la necesidad del cambio y qué partes del sistema afecta.

- Se recomienda establecer desde el inicio un canal de comunicación claro dentro de la comisión para coordinar la integración.
- El historial de commits es parte de la entrega y será revisado. Se espera que refleje un desarrollo progresivo a lo largo del tiempo — commits regulares, con mensajes descriptivos, realizados por el integrante responsable de cada repo. Un repo con un único commit o con todos los commits de la misma fecha es motivo de invalidación de la entrega.
- El uso de herramientas de inteligencia artificial (GitHub Copilot, ChatGPT, etc.) está **permitido**. Sin embargo, cada integrante es responsable de todo el código de su repositorio, independientemente de cómo fue generado. En la defensa se puede preguntar sobre cualquier parte del código — se espera que el alumno pueda explicarlo, justificar las decisiones tomadas y saber cómo modificarlo si fuera necesario.

Defensas

Cada etapa incluye una instancia de defensa. En cada una, la comisión dispondrá de un tiempo a definir previo a la defensa para presentar la entrega realizada y responder preguntas. Los horarios asignados a cada comisión se publicarán con anticipación.

La asistencia a la defensa es **obligatoria para todos los integrantes** de la comisión. Se espera que todos participen activamente y puedan explicar las decisiones tomadas, tanto las propias como las del equipo. No alcanza con que el sistema funcione: cada integrante debe poder dar cuenta de su trabajo y del diseño general del proyecto.